Test №1

1. Using only bare λ-calculus (variables, λ-abstraction and application), write down a λ-term for logical xor (exclusive OR) of two Church booleans. Remember that with Church booleans we have the following encoding:

$$\mathsf{tru} = \lambda t.\lambda f.t$$
$$\mathsf{fls} = \lambda t.\lambda f.f$$

2. Verify your implementation of xor by writing down evaluation sequence for the term xor tru tru.

3. Using only bare λ-calculus (variables, λ-abstraction and application), write down a λ-term for a function doubleThenInc that computes $2n + 1$ for any given Church numeral n. Remember that with Church numerals we have the following encoding:

$$\mathsf{c}_0 = \lambda s.\lambda z.z$$
$$\mathsf{c}_1 = \lambda s.\lambda z.sz$$
$$\mathsf{c}_2 = \lambda s.\lambda z.s(sz)$$
$$\mathsf{c}_3 = \lambda s.\lambda z.s(s(sz))$$
$$\ldots$$

You can verify your implementation by computing doubleThenInc $c_2\ f\ x \longrightarrow f\ (f\ (f\ (f\ (f\ x))))$.

Solution for Test №1

1. $\mathsf{xor} = \lambda x.\lambda y.x(y\ \mathsf{fls}\ \mathsf{tru})y$
2. $$\begin{aligned}
&\mathsf{xor}\ \mathsf{tru}\ \mathsf{tru} \\
&= (\lambda x.\ \lambda y.\ x\ (y\ \mathsf{fls}\ \mathsf{tru})\ y)\ \mathsf{tru}\ \mathsf{tru} \\
&\to (\lambda y.\ \mathsf{tru}\ (y\ \mathsf{fls}\ \mathsf{tru})\ y)\ \mathsf{tru} \\
&\to \mathsf{tru}\ (\mathsf{tru}\ \mathsf{fls}\ \mathsf{tru})\ \mathsf{tru} \\
&= (\lambda t.\ \lambda f.\ t)\ (\mathsf{tru}\ \mathsf{fls}\ \mathsf{tru})\ \mathsf{tru} \\
&\to (\lambda t.\ \lambda f.\ t)\ (\mathsf{tru}\ \mathsf{fls}\ \mathsf{tru})\ \mathsf{tru} \\
&\to (\lambda f.\ \mathsf{tru}\ \mathsf{fls}\ \mathsf{tru})\ \mathsf{tru} \\
&\to \mathsf{tru}\ \mathsf{fls}\ \mathsf{tru} \\
&= (\lambda t.\ \lambda f.\ t)\ \mathsf{fls}\ \mathsf{tru} \\
&\to (\lambda f.\ \mathsf{fls})\ \mathsf{tru} \\
&\to \mathsf{fls}
\end{aligned}$$
3. $\mathsf{doubleThenInc} = \lambda n.\lambda s.\lambda z.s(ns(nsz))$

Test №2

1. Using explicit recursion implement in Racket a function **alternating-sum** that computes a sum of a list of numbers, multiplying every second number by -1.
   For example, `(alternating-sum (list 6 2 4 1 3 9))` should compute $6-2+4-1+3-9=1$.

2. Consider the following definitions in Racket:

```
(define (dec n) (- n 1))

(define (f n)
  (cond
    [(<= n 2) (- 3 n)]
    [else (+ (f (dec n)) (f (dec (dec n))))]))
```

   Using the Substitution Model explain step-by-step how the following expression is computed (you can evaluate **cond**-expressions immediately, but evaluation of function calls to **f** and **dec** have to be explicit):

```
(f 3)
```

Solution for Test №2

1.
```
(define (alternating-sum lst)
  (cond
    [(empty? lst) 0]
    [else
     (- (first lst)
        (alternating-sum (rest lst)))]))
```

2.
```
(f 3)
= (+ (f (dec 3)) (f (dec (dec 3))))
= (+ (f 2) (f (dec (dec 3))))
= (+ 1 (f (dec (dec 3))))
= (+ 1 (f (dec 2)))
= (+ 1 (f 1))
= (+ 1 2)
= 3
```

Test №3

Consider the following sample definition:

```
(define employees
  '(("John" "Malkovich" . 29)
    ("Anna" "Petrova"   . 22)
    ("Ivan" "Ivanov"    . 23)
    ("Anna" "Karenina"  . 40)))
```

Recall that `'("Anna" "Petrova" . 22)` is equivalent to `(cons "Anna" (cons "Petrova" 22))`. This value is a pair where first element is the first name of an employee and second element is a pair of last name and age.

1. Implement a function `fullname` that takes employee and returns their full name as a pair of first and last name:

   ```
   (fullname '("John" "Malkovich" . 29))
   ; '("John" . "Malkovich")
   ```

2. Using higher-order functions (`map`, `ormap`, `andmap`, `filter`, `foldl`) and without explicit recursion, write down an expression that computes a list of entries from `employees` where employee's first name is `"Anna"`.

3. Using higher-order functions (`map`, `ormap`, `andmap`, `filter`, `foldl`) and without explicit recursion, implement a function `employees-over-25` that computes a list of full names of employees whose age is greater than 25 given a list of employee entries as input:

   ```
   (employees-over-25 employees)
   ; '(("John" . "Malkovich") ("Anna" . "Karenina"))
   ```

Solution for Test №3

1.
```
(define (fullname employee)
  (cons (car employee)
        (car (cdr employee))))
```

2.
```
(filter (lambda (employee)
          (equal? "Anna" (car employee)))
        employees)
```

3.
```
(define (employees-over-25 lst)
  (define (over-25? employee)
    (> (cdr (cdr employee)) 25))
  (map fullname (filter over-25? lst)))
```

Test №4

1. Using explicit recursion, implement a function `replicate` that takes a number `n` and a value `x`, and returns a list with `n` copies of `x`:

```
(replicate 10 'a)
; '(a a a a a a a a a a)
(replicate 3 '(1 . 2))
; '((1 . 2) (1 . 2) (1 . 2))
```

2. Using explicit recursion, implement a function `encode` that groups consecutive duplicate elements into pairs of the representative element and the length of the group:

```
(encode '(a a b c c c a a a))
; '((a . 2) (b . 1) (c . 3) (a . 3))
```

   Hint: first implement ***`prepend`*** *function that adds one more symbol at the beginning of an already encoded sequence.*

3. Using higher-order functions (`map`, `ormap`, `andmap`, `filter`, `foldl`) and without explicit recursion, implement a function `decode` that is the inverse of `encode`:

```
(decode '((a . 2) (b . 1) (c . 3) (a . 3)))
; '(a a b c c c a a a)
```

Solution for Test №4

1.
```
(define (replicate n x)
  (cond
    [(<= n 0) empty]
    [else
     (cons x (replicate (- n 1) x))]))
```

2.
```
(define (prepend x lst)
  (cond
    [(empty? lst)
     (list (cons x 1))]
    [(equal? x (car (first lst)))
     (cons (cons x (+ 1 (cdr (first lst))))
           (rest lst))]
    [else
     (cons (cons x 1)
           lst)]))

(define (encode lst)
  (cond
    [(empty? lst) empty]
    [else
     (prepend (first lst)
              (encode (rest lst)))]))
```

3.
```
(define (decode lst)
  (define (decode-pair p)
    (replicate (cdr p) (car p)))
  (apply append (map decode-pair lst)))
```

Test №5

1. Using explicit recursion, implement function `propagate :: (Bool, [Int]) -> [(Bool, Int)]` that pairs given boolean with every integer in a given list:

```
example1 = propagate (False, [1, 2, 3])
-- [(False,1),(False,2),(False,3)]
example2 = propagate (True, [1, 1])
-- [(True,1),(True,1)]
```

2. Consider the following definitions:

```
data Radians = Radians Double
data Degrees = Degrees Double

pi :: Double
pi = 3.14159
```

   Implement the following functions that convert between degrees and radians:

```
toDegrees :: Radians -> Degrees
fromDegrees :: Degrees -> Radians
```

Solution for Test №5

1. ```
   propagate :: (Bool, [Int]) -> [(Bool, Int)]
   propagate (b, [])   = []
   propagate (b, x:xs) = (b, x) : propagate (b, xs)
   ```

2. ```
   toDegrees :: Radians -> Degrees
   toDegrees (Radians x) = Degrees (x * 180 / pi)

   fromDegrees :: Degrees -> Radians
   fromDegrees (Degrees x) = Radians (x * pi / 180)
   ```

Test №6

Consider the following definitions:

```haskell
type Name = String
data Grade = A | B | C | D
data Student = Student Name Grade

data Result a
    = Success a
    | Failure String
```

1. Using explicit recursion, implement function `studentsWithA :: [Student] -> [Name]` that returns a list of names of students with `A` grade:

   ```haskell
   example1 = studensWithA [Student "Jack" D, Student "Jane" A]
   -- ["Jane"]
   ```

   Note: you cannot use `(==)` to compare grades for equality.

2. Implement a polymorphic higher-order function

   ```haskell
   combineResultsWith :: (a -> b -> c) -> Result a -> Result b -> Result c
   ```

   that applies a function to two results if both are present and returns the first available error message otherwise:

   ```haskell
   example2 = combineResultsWith (+) (Success 2) (Success 3)
   -- Success 5
   example3 = combineResultsWith (+) (Failure "x is undefined") (Failure "crash")
   -- Failure "x is undefined"
   ```

Solution for Test №6

1. ```
   studentsWithA :: [Student] -> [Name]
   studentsWithA [] = []
   studentsWithA (Student name A : students)
     = name : studentsWithA students
   ```

2. ```
   combineResultsWith
     :: (a -> b -> c)
     -> Result a
     -> Result b
     -> Result c
   combineResultsWith f (Success x) (Success y) = Success (f x y)
   combineResultsWith f (Failure msg) _ = Failure msg
   combineResultsWith f _ (Failure msg) = Failure msg
   ```

Test №7

Consider the following definitions:

1. (1 point)
   What is the type of `guess` in the following program?

```
guess p g = do
  s <- getLine
  x <- g s
  case p x of
    True  -> return x
    False -> guess p g
```

2. (1 point)
   Implement a program `echo :: IO ()` that goes through an infinite loop of reading user input and printing it back in CAPS. Use `toUpper :: Char -> Char` to convert a single character to upper case. Remember that `type String = [Char]`.

```
import Data.Char (toUpper)

echo :: IO ()
```

3. (+1 extra credit)
   Implement a polymorphic function `forIO_` that runs a program for each element in a given list (using given function):

```
example = do
  forIO_ [1, 2] (\n ->
    forIO_ "ab" (\c ->
      putStrLn (replicate n c)))

-- When executed, example prints out:
-- a
-- aa
-- b
-- bb
```

Solution for Test №7

1. ```haskell
   guess :: (a -> Bool) -> (String -> IO a) -> IO a
   ```

2. ```haskell
   echo :: IO ()
   echo = do
     input <- getLine
     putStrLn (map toUpper input)
     echo
   ```

3. ```haskell
   -- solution 1
   forIO_ :: [a] -> (a -> IO ()) -> IO ()
   forIO_ xs f = mapM_ f xs

   -- solution 2
   forIO_ :: [a] -> (a -> IO ()) -> IO ()
   forIO_ xs f = sequence_ (map f xs)

   -- solution 3
   forIO_ :: [a] -> (a -> IO ()) -> IO ()
   forIO_ [] f = return ()
   forIO_ (x:xs) f = do
     f x
     forIO_ xs f
   ```
   ```

Test №8

1. (1 point)
   What is the type of `guess` in the following function?

   ```
   f x y z = x (y : x z)
   ```

2. (1 point)
   Implement a polymorphic function `repeatUntil :: (a -> Bool) -> (a -> IO a) -> a -> IO a`. Expression `repeatUntil p f x` should repeat application of `f` to `x` until predicate `p` is satisfied. Example usage:

   ```
   tryGetInt :: IO (Maybe Int)
   tryGetInt = do
     line <- getLine
     return (readMaybe line)

   example1 = repeatUntil (< Just 10) (\x -> tryGetInt) Nothing
   -- asks user for input until the user provides a number less than 10

   example2 = repeatUntil (< 10) (\x -> return 5) 15
   -- returns 5
   ```

3. (1 point)
   Consider the following data type, corresponding to possible input events:

   ```
   type Point = (Double, Double)
   data Event = Click | Move Point
   ```

   Implement function `toPolyline :: Event -> [Point]` that returns a list of points where the user has clicked.

   ```
   example3 = toPolyline [Move (1, 2), Move (3, 4), Click, Move (5, 6), Click, Click]
   -- [(3, 4), (5, 6), (5, 6)]
   ```

4. (+1 extra credit)
   Implement function `toPolyline' :: Event -> [Point]` that returns a list of points where the user has clicked, without duplicate consecutive clicks:

   ```
   example4 = toPolyline' [Move (1, 2), Move (3, 4), Click, Move (5, 6), Click, Click]
   -- [(3, 4), (5, 6)]
   ```

5. (1 point)
   Implement function `prefixes :: [a] -> [[a]]` that returns a list of all prefixes of the given list. The function should be lazy and support infinite lists:

   ```
   example5 = prefixes "hello"
   -- ["","h","he","hel","hell","hello"]
   example6 = prefixes [1, 2, 3]
   -- [[],[1],[1,2],[1,2,3]]
   example7 = take 4 (prefixes [1..])
   -- [[],[1],[1,2],[1,2,3]]
   ```

Solution for Test №8

1.
```haskell
f :: ([a] -> [a]) -> a -> [a] -> [a]
```

2.
```haskell
repeatUntil :: (a -> Bool) -> (a -> IO a) -> a -> IO a
repeatUntil p f x
  | p x = return x
  | otherwise = repeatUntil p f (f x)
```

3.
```haskell
toPolyline :: [Event] -> [Point]
toPolyline = helper Nothing
  where
    helper _ (Move point : events) = helper (Just point) events
    helper Nothing (Click : events) = helper Nothing events
    helper (Just point) (Click : events) = point : helper (Just point) events
```

4.
```haskell
data IsUsed = Used | NotUsed

toPolyline' :: [Event] -> [Point]
toPolyline' events = removeDuplicates (toPolyline events)
  where
    removeDuplicates :: [Point] -> [Point]
    removeDuplicates [] = []
    removeDuplicates (x:xs) = x : removeDuplicates (dropWhile (== x) xs)
```

5.
```haskell
prefixes :: [a] ->[[a]]
prefixes [] = [[]]
prefixes (x:xs) = [] : map (x:) (prefixes xs)
```

Test №9

1. (1 point)
   What is the type of `k` in the following declaration?

   ```
   k g = map (\x -> putStrLn (g x))
   ```

2. (1 point)
   Given the following definition of `Result`, write down a polymorphic function

   ```
   splitResults :: [Result a] -> ([String], [a])
   ```

   that splits results into `Failure`s and `Success`es:

   ```
   data Result a = Success a | Failure String

   divide :: Int -> Int -> Result Int
   divide _ 0 = Failure "division by zero"
   divide n m = Success (n `div` m)

   example2 = splitResults (zipWith divide [6, 7, 8] [3, 0, 2])
   -- (["division by zero"],[2,4])
   ```

3. (1 point)
   Consider the following data type, representing a 2D grid:

   ```
   data Grid a = Grid [[a]]
   ```

   Implement function `mapGrid :: (a -> b) -> Grid a -> Grid b` that applies a given function to every element in the grid.

4. (+1 extra credit)
   Implement function `enumerateGrid :: Grid a -> Grid (Int, a)` that enumerates all elements in a given grid, line by line, from top to bottom and from left to right:

   ```
   example4 = enumerateGrid (Grid [['a', 'b'], ['c', 'd']])
   -- Grid [[(1,'a'),(2,'b')],[(3,'c'),(4,'d')]]
   ```

Solution for Test №9

1.
```haskell
k :: (a -> String) -> [a] -> [IO ()]
k g = map (\x -> putStrLn (g x))
```

2.
```haskell
splitResults :: [Result a] -> ([String], [a])
splitResults [] = ([], [])
splitResults (result : results) =
  case result of
    Success x   -> (failures, x : successes)
    Failure err -> (err : failures, successes)
  where
    (successes, failures) = splitResults results
```

3.
```haskell
mapGrid :: (a -> b) -> Grid a -> Grid b
mapGrid f (Grid rows) = Grid (map (map f) rows)
```

4.
```haskell
enumerateGrid :: Grid a -> Grid (Int, a)
enumerateGrid (Grid rows) = Grid (enumerateRowsFrom 1 rows)
  where
    enumerateRowsFrom _ [] = []
    enumerateRowsFrom n (row:moreRows) = row' : moreRows'
      where
        (n2, row') = enumerateRowFrom n row
        moreRows'  = enumerateRowsFrom n2 moreRows

    enumerateRowFrom n [] = (n, [])
    enumerateRowFrom n (x:xs)
      = (n2, (n, x) : ys)
      where
        (n2, ys) = enumerateRowFrom (n+1) xs
```

Test №10

Consider the following knowledge base:

```
% student(Name, Group)
student(alisa,  2).
student(bob,    1).
student(chloe,  2).
student(denise, 1).
student(edward, 2).

friend(alisa, bob).
friend(alisa, denise).
friend(bob, chloe).
friend(bob, edward).
friend(chloe, denise).
friend(denise, edward).

unary(z).
unary(s(X)) :- unary(X).
```

1. (1 point)
   Write down predicate `groupmates/2` that checks whether two students are from the same group.

   ```
   ?- groupmates(alisa, bob)
   false

   ?- groupmates(alisa, edward)
   true
   ```

2. (1 point)
   Draw the search tree for the following query

   ```
   ?- friend(alisa, Y), friend(Y, Z).
   ```

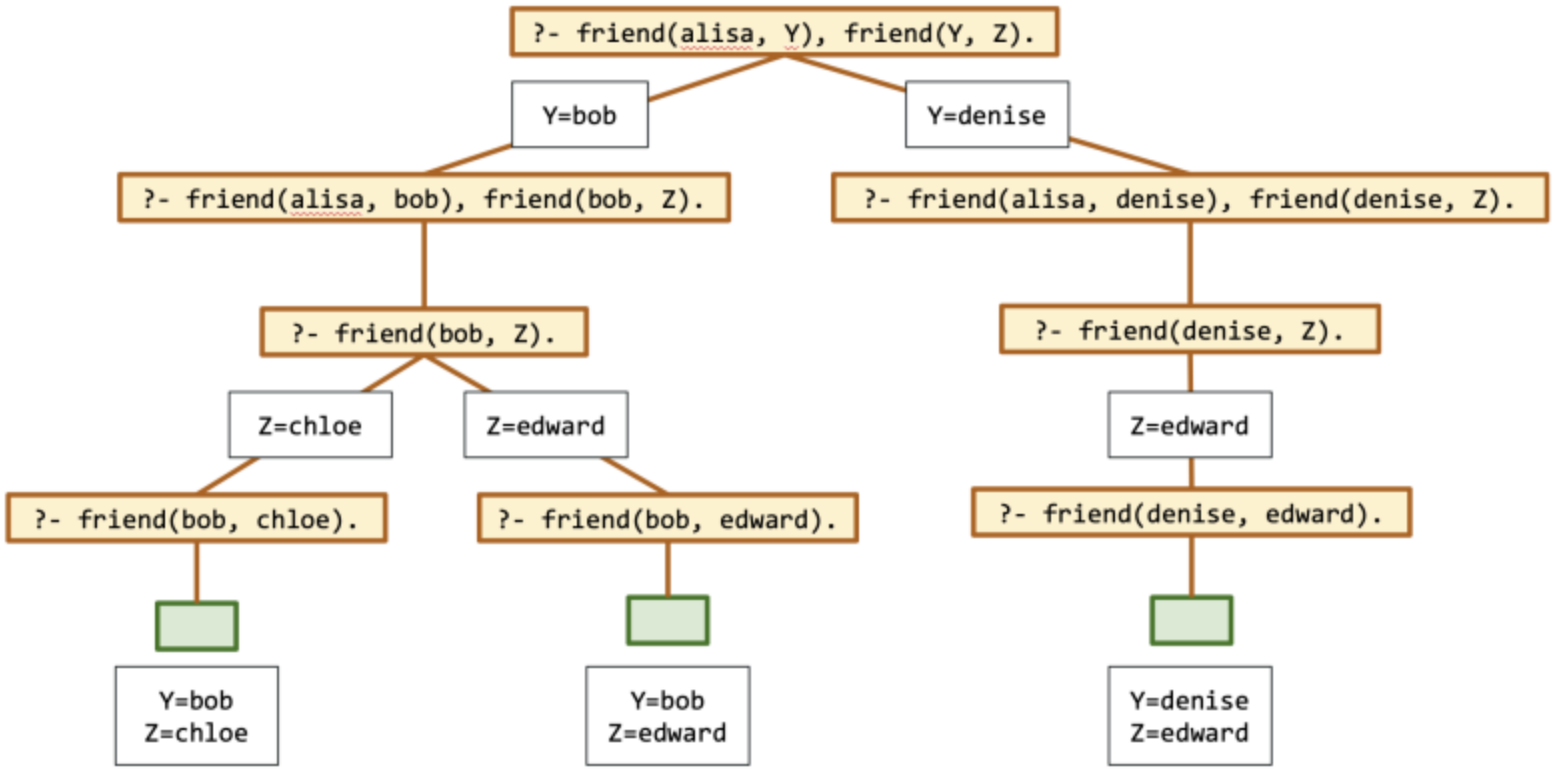
3. (+1 extra credit)
   Implement a multiplication of unary numbers as a predicate `mult/2`:

   ```
   ?- mult(s(s(z)), s(s(s(z))), X)
   X = s(s(s(s(s(s(z))))))

   ?- mult(X, s(s(s(z))), s(s(s(s(s(s(z)))))))
   X = s(s(z))
   ```

Solution for Test №10

1.
```
groupmates(Student1, Student2) :-
    student(Student1, Group),
    student(Student2, Group).
```



2.

3.
```
add(z, Y, Y).
add(s(X), Y, s(Z)) :- add(X, Y, Z).

mult(z, _, z).
mult(s(X), Y, Z) :- mult(X, Y, R), add(Y, R, Z).
```

Test №11

Consider the following knowledge base:

1. (1 point)
   Write down predicate `sublist/2` that checks whether the first list is a sublist of the second one.

   ```
   ?- sublist([2, 3, 5], [1, 2, 3, 4, 5])
   true

   ?- sublist(X, [1, 2])
   X = [1, 2]
   X = [1]
   X = [2]
   X = []
   ```

2. (1 point)
   Implement predicate `search/3` that checks whether the first list occurs in the second list at a given position:

   ```
   ?- search([a,b,a], [c,a,b,a,b,a,d], Pos)
   Pos = 1
   Pos = 3

   ?- search(Needle, [c,a,b,a,b,a,d], 5)
   Needle = []
   Needle = [a]
   Needle = [a, d]
   ```

3. (+1 extra credit)
   Implement predicate `replace/4`. `replace(Old, New, OldWhole, NewWhole)` checks whether `NewWhole` can be constructed from `OldWhole` by replacing some occurrences of `Old` with `New`:

   ```
   ?- replace([a,b], [x,y,z], [a,b,r,a,b,a], L)
   L = [x, y, z, r, x, y, z, a]
   L = [x, y, z, r, a, b, a]
   L = [a, b, r, x, y, z, a]
   L = [a, b, r, a, b, a]

   ?- replace([a,a], [x,y], [a,a,a,a], L)
   L = [x, y, x, y]
   L = [x, y, a, a]
   L = [a, x, y, a]
   L = [a, a, x, y]
   L = [a, a, a, a]
   ```

Solution for Test №11

1. ```
   sublist([], L).
   sublist([H|T], [H|L]) :- sublist(T, L).
   sublist(T, [_|L]) :- sublist(T, L).
   ```

2. ```
   prefix([], _).
   prefix([H|T], [H|L]) :- prefix(T, L).

   search(Needle, Haystack, 0) :- prefix(Needle, Haystack).
   search(Needle, [_|Haystack], N) :- search(Needle, Haystack, K), N is K+1.
   ```

3. ```
   replace(_Old, _New, [], []).
   replace(Old, New, OldWhole, NewWhole) :-
     append(Old, OldSuffix, OldWhole),
     replace(Old, New, OldSuffix, NewSuffix),
     append(New, NewSuffix, NewWhole).
   replace(Old, New, [H|OldWhole], [H|NewWhole]) :-
     replace(Old, New, OldWhole, NewWhole).
   ```
   ```

Test №12

Consider the following knowledge base:

1. (1 point)
   Write down predicate `minimum/2` that finds the minimum number in a list of numbers.
   Hint: use a helper function.

   ```
   ?- minimum([3, 6, 2, 5, 4, 7], X)
   X = 2
   ```

2. (1 point)
   Without using negation or `fail`, implement predicate `remove/3` that removes all occurrences of a given element from a list:

   ```
   ?- remove(e, [a,p,p,l,e, p,i,e], X)
   X = [a, p, p, l, p, i]
   ```

3. (1 point)
   Implement predicate `removeU/3` that removes all elements from a list that can be unified with a given term:

   ```
   ?- removeU(t(X), [1, a, A, tb, t(b), tX, t(B)], Y)
   Y = [1, a, tb, tX]
   ```

4. (1 point)
   Consider predicates `nat/1` and `nat/2` defined as follows:

   ```
   nat(0).
   nat(N) :- nat(K), N is K+1.

   nat(0, 0) :- !.
   nat(0, Max) :- Max > 0.
   nat(N, Max) :- M is Max-1, nat(K, M), N is K+1.
   ```

   Implement predicate `prime/1` that checks if a given number is a prime number. The implementation should encode the following definition: "a number N is prime if there there does not exist a pair of numbers $1 < X, Y < N$ such that $X \times Y = N$."

   ```
   ?- prime(29)
   true
   ?- prime(N)
   N = 2
   N = 3
   N = 5
   ...
   ```

5. (+1 extra credit)
   Implement predicate `allColors/1` that is true whenever a given list consists of all distinct colors defined by predicate `color`:

   ```
   color(red). color(green). color(blue).
   ```

   Sample queries:

   ```
   ?- allColors([red, green, blue])
   true
   ?- allColors([red, G, blue])
   G = green
   ?- allColors(Colors)
   Colors = [blue, green, red]
   ```

   Hint: a list of all colors is a list of distinct colors such that there does not exist a color that is not in that list.

Solution for Test №12

1. ```
   % using a helper predicate with accumulator parameter
   minimum([H|T], X) :- minAcc(T, H, X).

   minAcc([], CurrentMin, CurrentMin).
   minAcc([H|T], CurrentMin, Min) :-
     H > CurrentMin,
     minAcc(T, CurrentMin, Min).
   minAcc([H|T], CurrentMin, Min) :-
     H =< CurrentMin,
     minAcc(T, H, Min).

   % using a check that head is smallest
   minimum([H|T], H) :- smallest(H, T).
   minimum([_|T], X) :- minimum(T, X).

   smallest(X, []).
   smallest(X, [H|T]) :- X < H, smallest(X, T).

   % using recursive call and a cut
   minimum([X], X).
   minimum([H|T], X) :- minimum(T, X), X < H, !.
   minimum([H|T], H).

   % using member predicate and negation:
   % minimum is an element of the list
   % such that no other element of the list is smaller
   minimum(L, X) :- member(X, L), \+ (member(Y, L), Y < X).
   ```

2. ```
   remove(X, [Y|Before], After) :-
     X = Y, !, remove(X, Before, After).
   remove(X, [Y|Before], [Y|After]) :-
     remove(X, Before, After).
   ```

3. ```
   % using double negation to check unification without keeping its effects
   removeU(X, [Y|Before], After) :-
     \+ (\+ X = Y), !, removeU(X, Before, After).
   removeU(X, [Y|Before], [Y|After]) :-
     removeU(X, Before, After).
   ```

4. ```
   prime(N) :- nat(N), N > 1, \+ composite(N).

   composite(N) :-
     nat(X, N), 1 < X, X < N,
     nat(Y, N), 1 < Y, Y < N,
     N is X * Y.
   ```

5. ```
   distinctColors([]).
   distinctColors([H|T]) :-
     distinctColors(T),
     color(H),
     \+ member(H, T).

   allColors(L) :-
     distinctColors(L),
     \+ (color(C), \+ member(C, L)).
   ```
   ```